

03_DUAL_OPERATIONAL_HARNESS_AND_MCP_SPEC.md

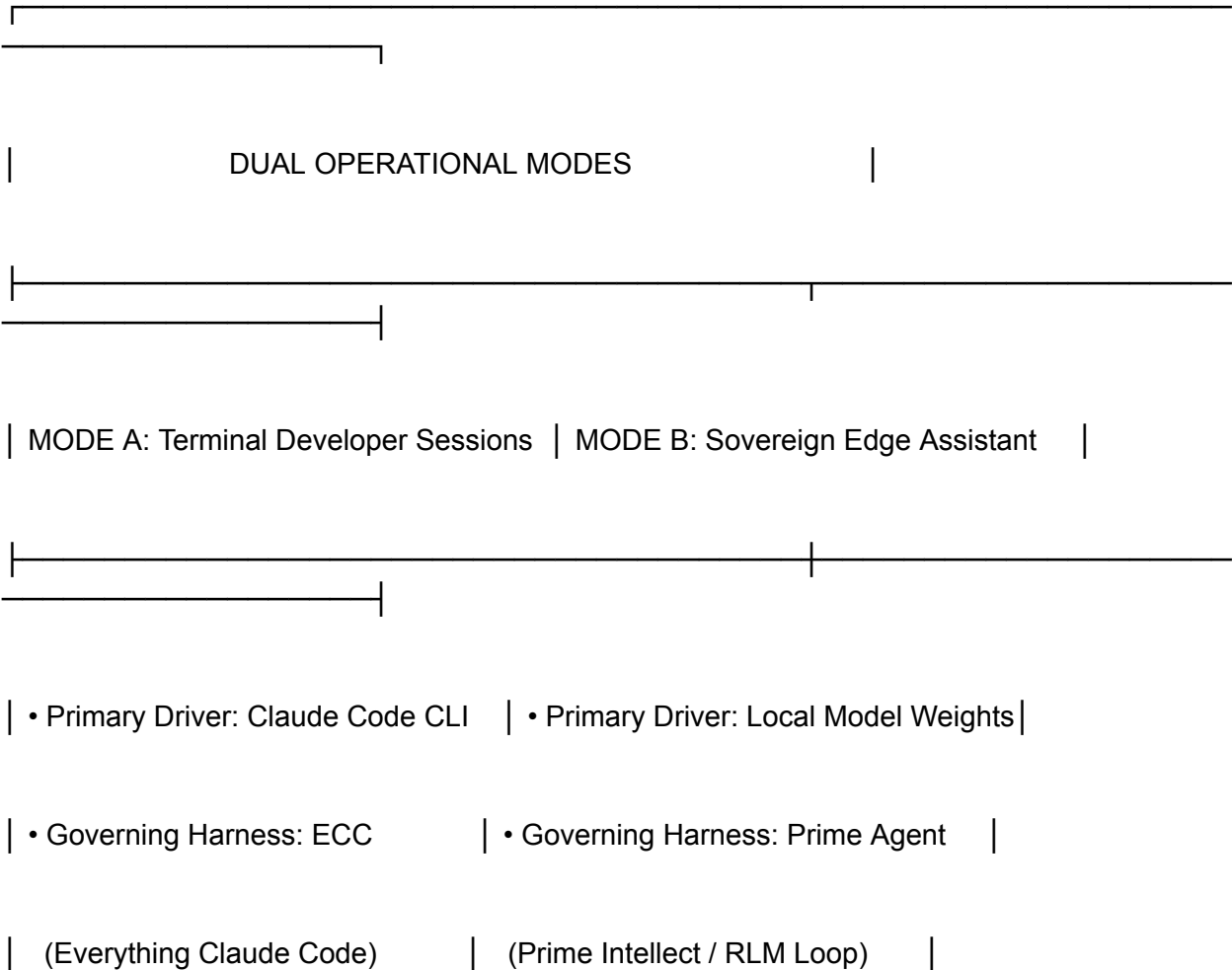
Dual Operational Modes, Prime Agent RLM Harness, MCP Infrastructure & AST Code Graphs

1. Document Scope & Supersession

This specification defines the execution loops, model harnesses, MCP server deployments, and code-graph analytical pipelines. It envelops, unifies, and supersedes:

- * master-execution-router
- * skill_manifest.json
- * Data dump and skill extraction.
- * Tool harness specifications in 5- Potential.34.ARCH.MATRIX-(5-S.F.'s-34-files)

2. Dual Operational Modes: Complete Separation of Concerns



| • Operational Arena: Terminal CLI | • Operational Arena: Snapdragon NPU |

| • Core Skill Suite: honey-crush, | • Loop Mechanism: Python REPL |

| nexus-mapper, ecc-planner, Pocock | Continual Harness (/refine) |

|
|

|

| (When Edge Assistant invokes Claude Code)

▼

|
|

| DISCRETE SUBPROCESS INVOCATION (Zero Nested Wrapping) |

| - Prime Agent calls Claude Code CLI strictly as an external utility. |

| - Claude Code executes independently, returns output text, and terminates. |

| - No nested harness wrapping or state conflicts. |

|
|

Beginner-Proof Explanation:

* Mode A (Claude Code CLI + ECC): When you are actively sitting at a terminal typing code

commands, Claude Code is your harness. ECC gives it specialized skills like honey-crush (reading massive docs) and nexus-mapper (mapping code files). Prime Agent is completely off.

* Mode B (Edge Assistant + Prime Agent): When you are on your phone using voice or screen vision, local open weights (Qwen 3.5) run on your Snapdragon NPU. Prime Agent (from Prime Intellect) runs this loop inside Python. It treats context as variables and tools as callable Python functions.

* The Bridge: If your on-device Prime Agent needs Claude to solve a complex coding task, it simply runs `claude --print "fix this bug"` as a command-line tool. It captures the answer and continues its day. They never fight over who owns the session.

3. What Prime Agent Operates Off Of

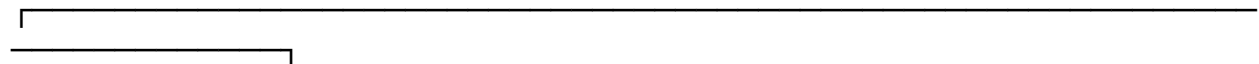
According to Prime Intellect's architecture:

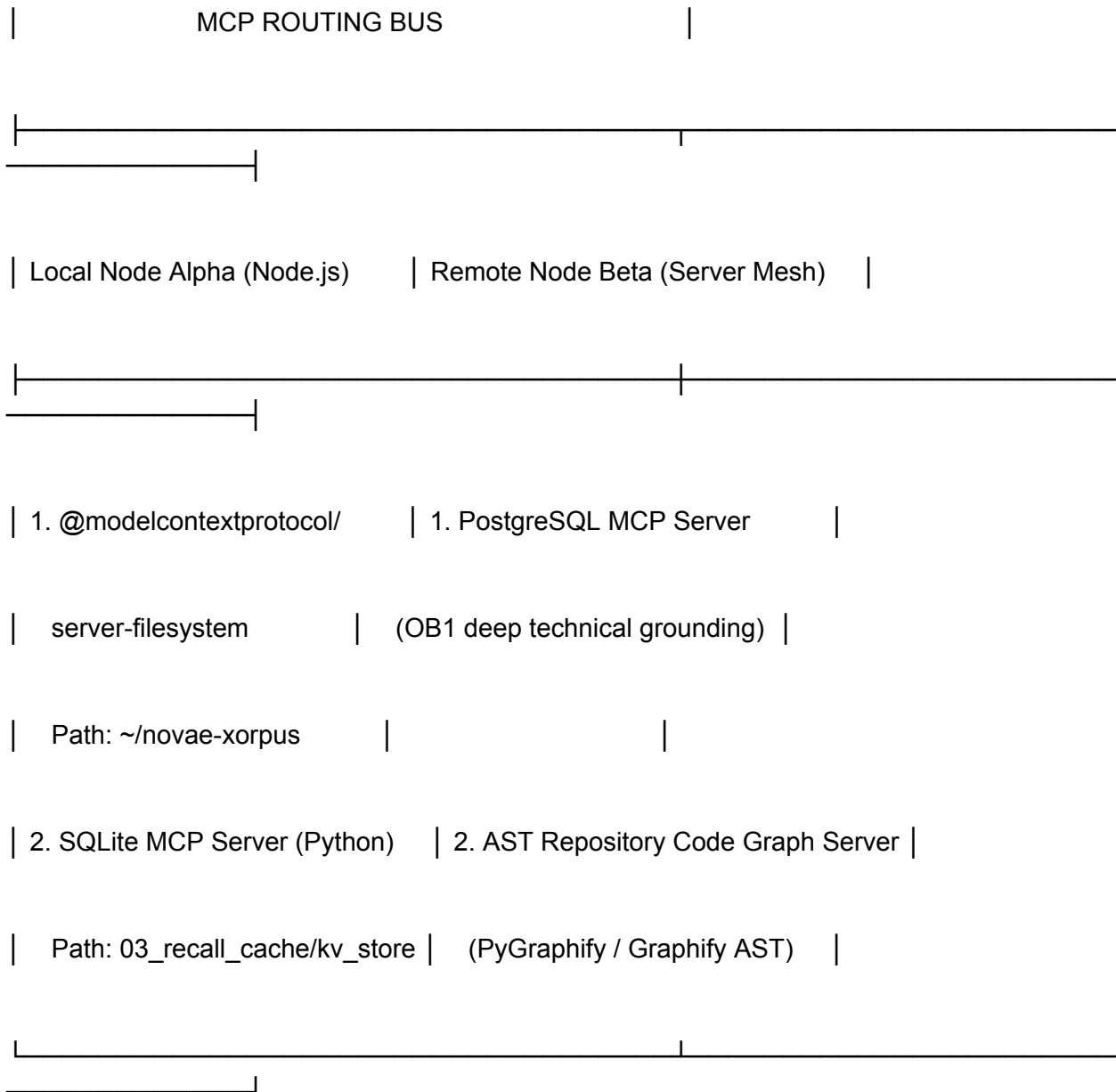
1. Persistent Python / IPython Kernel: Tools are loaded into the Python namespace as functions. The agent writes small code blocks to call them instead of massive JSON schema handoffs.
2. OpenAI-Compatible Client: Prime Agent connects to an HTTP endpoint. It routes through OmniRoute on `http://localhost:20128/v1`, which handles prompt caching and token conservation before forwarding to GenieX on the NPU.
3. The Continual Harness: A state layer that refines supplemental prompts and skill references via `/refine` with automated rollback snapshots.

4. Model Context Protocol (MCP) Server Infrastructure

[Horizons UI / Prime Agent / Claude Code]

|





* Filesystem MCP: Runs via Node.js LTS in the background:

```
npx -y @modelcontextprotocol/server-filesystem ~/novae-xorpus
```

Exposes the vault safely to agents without granting access to root Android system files.

5. Code Review Graph, Knowledge Graph & NotebookLM Pipeline

1. PyGraphify / Graphify: Ingests raw code repositories, parses Abstract Syntax Trees (ASTs), and maps which functions call which classes.
2. OpenWiki TUI: Writes these relationships into clean Markdown notes inside 02_wiki_md/ with bidirectional links ([[class_a]] <-> [[class_b]]).
3. High-Speed Cache: Dumps pre-tokenized chunks into 03_recall_cache/jsonl/ and SQLite tables for instant retrieval.
4. NotebookLM (notebooklm-py): Serves as the deep analytical engine for multi-document queries without flooding the prompt context window.